autolisp-file-compat Design

System Administrator

May 25, 2026

Contents

1 Purpose **1**

2 Design Goals **1**

3 Initial Scope **2**

4 Scenario Model **2**

5 Artifact Model **3**

6 Comparison Model **4**

7 Classification **4**

8 Report Model **4**

9 Runner Kinds **5**

10 Future Runners **5**

1 Purpose

This module provides a dedicated compatibility-audit harness for file, stream, pathname, and printer behavior relevant to AutoLISP and Visual LISP.

The goal is not just to run smoke tests, but to preserve enough structured evidence to compare:

- local `clautolisp` behavior,

- host Common Lisp behavior where it matters,
- later real-product behavior under AutoCAD, BricsCAD, or other targets.

2 Design Goals

- Keep file compatibility auditing separate from ordinary unit tests.
- Capture exact bytes as well as decoded text.
- Make newline and encoding handling explicit instead of accidental.
- Preserve scenario metadata and result classification.
- Support multiple runners over the same scenario corpus.

3 Initial Scope

The first implementation supports a local runner only.

That runner is responsible for:

- creating a scenario workspace,
- writing text or byte fixtures,
- reading them back,
- capturing byte, text, and line artifacts,
- comparing actual and expected results,
- producing a structured report.

4 Scenario Model

Each scenario should eventually include:

- stable name,
- description,
- runner identifier,

- workspace root,
- relative file paths,
- encoding and newline controls,
- steps or operations,
- comparison expectations,
- result classification.

The first implementation keeps the executable surface small and uses simple round-trip scenarios with explicit expected text or bytes.

The current implementation also supports a second scenario kind for builtin execution. Those scenarios:

- create a temporary workspace,
- install setup files and directories,
- configure current, support, and trusted path state,
- call a selected builtin through the runtime function-object layer,
- compare the resulting runtime value,
- optionally capture a post-operation artifact.

The current scenario corpus is now organized as a directory tree rather than a single flat list of files. This makes it easier to grow focused scenario families such as:

- newline handling,
- encoding handling,
- later pathname and search-path behavior,
- later printer and file-mutation behavior.

5 Artifact Model

Each executed scenario may produce artifacts such as:

- absolute path of the file under test,
- raw bytes,
- decoded text,
- decoded lines,
- metadata such as encoding and newline mode.

Artifacts are part of the compatibility evidence and should be preserved in the report structure even when the scenario succeeds.

Builtin scenarios may legitimately have no captured artifact when the function under test returns a value but does not leave a file result behind.

6 Comparison Model

Comparisons are intentionally separated by layer:

- byte comparison,
- text comparison,
- line comparison.
- runtime-value comparison for builtin results.

This matters because some scenarios are expected to be text-equivalent while not being byte-equivalent, and others must match exactly at the byte level.

7 Classification

Scenarios and checks should eventually be classifiable as:

- portable,
- implementation-sensitive,

- host-sensitive,
- product-sensitive,
- unknown.

The implementation now records scenario-level classification metadata directly in the scenario files and preserves it through report emission and filtering.

8 Report Model

Reports should be useful both per-scenario and in aggregate.

The current implementation therefore emits:

- individual scenario reports,
- aggregate summary counts for scenarios and checks.

This keeps machine-readable runs suitable for CI and later diff-oriented audit workflows.

9 Runner Kinds

The harness now supports:

- local `clautolisp` runner,
- SBCL helper runner,
- CCL helper runner,
- product-adapter runners for AutoCAD and BricsCAD through external runner templates.

The product-adapter runners currently execute generated AutoLISP wrapper files and read back a result s-expression for the scenario final value. This is intentionally narrower than the local runner: it is aimed at executable product audits of builtin/file behavior rather than full introspective debugging.

10 Future Runners

Planned refinements include:

- richer product-side step reporting,
- better support for host-state-sensitive scenarios such as search paths,
- additional CAD environments beyond AutoCAD and BricsCAD.

Those runners should share the same scenario and report model as much as possible.

The current local runner already provides two execution styles over that shared model:

- round-trip file scenarios,
- builtin-execution scenarios.